

Data Collection Techniques

Web Scraping for the Data Science Pipeline

Pipeline Covered in These Notes

`requests` (fetch raw HTML) → `BeautifulSoup`
(parse & extract)
→ `pandas` (structure data) → `.csv` (store
data)

Kabir Roy

VIT Bhopal University

Computer Science Engineering

Compiled for Data Science interview & project prep

June 30, 2026

Contents

1	Introduction: Why Web Scraping?	2
2	The requests Module	3
2.1	Installing and Importing	3
2.2	Fetching a Single Page	3
2.3	Checking the Response Status	4
2.4	Saving HTML to a File	4
2.5	Downloading Many Pages in a Loop	5
3	The BeautifulSoup Module	5
3.1	Installing and Creating a Soup Object	6
3.2	Reading Saved HTML and Parsing It	7
3.3	Accessing Elements — Dot Notation	7
3.4	find() — One Match, With Filters	7
3.5	find_all() — All Matches	8
3.6	select() and select_one() — CSS Selectors	8
3.7	Extracting Attribute Values	8
3.8	Traversing the Tree	9
3.9	Handling Missing Elements Safely	9
4	Full Mini Project: Scraping Book Data to CSV	9
4.1	Step 1: Load the Saved Page and Build Soup	10
4.2	Step 2: Select Every Book Block on the Page	10
4.3	Step 3: Loop Over Each Book and Extract Fields	10
4.4	Step 4: Structure the Data with pandas	11
4.5	Step 5: Save to CSV	11
5	Extras: Beyond the Notebooks	12
5.1	Headers and User-Agents	12
5.2	Query Parameters	12
5.3	Handling Request Failures Gracefully	13
5.4	Being a Polite Scraper: Rate Limiting	13
5.5	Pagination Patterns	14
5.6	When to Use an API Instead	14
5.7	Dynamic (JavaScript-Rendered) Pages	14
5.8	Quick Reference Table	15

1 Introduction: Why Web Scraping?

What is Data Collection in Data Science?

Before any analysis, modeling, or machine learning can happen, you need **data**. Sometimes data comes ready-made as a CSV or a database. Often, however, the data you need lives on a website — product listings, news articles, prices, reviews — and there is no clean API for it. **Web scraping** is the technique of programmatically downloading and extracting that data directly from web pages.

The Two-Library Pipeline

Web scraping in Python almost always uses two libraries together, each with one job:

- **requests** — talks to the internet. It sends an HTTP request to a URL and brings back the raw HTML (just text, like a giant string).
- **BeautifulSoup** — talks to the HTML. It takes that raw text and turns it into a searchable tree structure, so you can pull out exactly the tags, classes, or attributes you want.

Think of it like ordering food: **requests** is the delivery person who brings the sealed box to your door (the raw HTML page). **BeautifulSoup** is you, opening the box and picking out the specific items you actually want to eat (the data fields).

Practice Sites Used in These Notes

Two safe, scraping-friendly demo sites are used throughout:

- <https://books.toscrape.com/> — a fake bookstore, perfect for practicing product-listing scrapes (titles, prices, ratings).
- <https://quotes.toscrape.com/> — a fake quotes site, good for text/author scraping practice.

Both are explicitly built for scraping practice, so there is no ethical or legal concern in using them to learn.

Scraping Ethics — Always Check First

Before scraping any *real* website (not a practice site):

- Check the site's `robots.txt` file (e.g. <https://example.com/robots.txt>) to see what the site allows automated bots to access.
- Check the site's Terms of Service — many explicitly forbid scraping.
- Don't hammer a server with too many rapid requests — add delays (`time.sleep()`) between requests to avoid overloading it or getting your IP banned.
- Prefer an official API if the site offers one — it's more stable and more polite than scraping HTML.

The Full Pipeline Covered in These Notes

1. **Fetch:** Use `requests.get()` to download the raw HTML of a page.
2. **Parse:** Feed that HTML into `BeautifulSoup` to build a searchable tree.
3. **Extract:** Use methods like `.find()`, `.find_all()`, and `.select()` to pull out the specific pieces of data you need.
4. **Structure:** Load the extracted data into a `pandas DataFrame`.
5. **Store:** Save the `DataFrame` to a `.csv` file for later analysis.

2 The requests Module

What is requests?

requests is a Python library used to send HTTP requests easily. It lets you fetch the content of a webpage programmatically — exactly what your browser does when you type a URL and hit Enter, except now Python does it and hands you the raw result as text. It is almost always the **first step** before parsing HTML with BeautifulSoup.

2.1 Installing and Importing

Install

```
1 pip install requests
```

Line-by-Line Explanation

- This installs the **requests** library from PyPI (Python’s package index).
- It only needs to be run **once** per environment — after that, the library is available to import in any script or notebook.

Import

```
1 import requests
```

Line-by-Line Explanation

- Brings the installed library into your current Python session so you can call its functions, like **requests.get(...)**.

2.2 Fetching a Single Page

Basic GET Request

```
1 url = "https://example.com"
2 response = requests.get(url)
3
4 print(response.text)
```

Line-by-Line Explanation

- **Line 1:** Stores the target web address as a string in the variable **url**.
- **Line 2:** **requests.get(url)** sends an HTTP **GET** request to that address (a GET request just means “give me this page”). The entire server reply — status code, headers, and HTML body — is captured in the **response** object.
- **Line 3:** **response.text** is an attribute holding the page’s HTML content as one big string. Printing it dumps the raw HTML to the screen, exactly as the server sent it (before any browser rendering).

2.3 Checking the Response Status

Status Code

```
1 print(response.status_code)
```

Line-by-Line Explanation

- Every HTTP response carries a numeric **status code** telling you whether the request succeeded. `response.status_code` returns that number (e.g. 200).
- Always check this before trying to parse the page — if the request failed, the HTML you got back might be an error page, not real content.

Common Status Codes

- **200** — OK (success, page fetched correctly)
- **403** — Forbidden (server is blocking you, often due to missing headers or anti-bot protection)
- **404** — Not Found (the URL/page doesn't exist)
- **500** — Internal Server Error (something broke on the server's end)

Good Practice — Validating Before Using

```
1 if response.status_code == 200:
2     print("Page fetched successfully!")
3 else:
4     print("Failed to fetch the page.")
```

Line-by-Line Explanation

- **Line 1:** Checks whether the status code equals 200 (success).
- **Line 2:** If true, prints a confirmation — it is now safe to proceed and parse `response.text`.
- **Lines 3–4:** If the code is anything else, prints a failure message instead of silently trying (and failing) to parse broken or missing content.

2.4 Saving HTML to a File

Writing Raw HTML to Disk

```
1 a = requests.get("https://books.toscrape.com/")
2
3 with open(f"htmls/page1.html", "w") as f:
4     f.write(a.text)
```

Line-by-Line Explanation

- **Line 1:** Fetches the homepage of the practice bookstore site and stores the response object in `a`.
- **Line 3:** `open(..., "w")` opens (or creates) a file in **write mode**. The `with` block automatically closes the file once you're done, even if an error occurs — this is the safe, standard way to handle files in Python.
- The `f"..."` is an f-string; here it's just used as a normal string since there's no variable inside the braces yet — but it sets up the pattern for the next example where the filename does

change per loop.

- **Line 4:** `f.write(a.text)` writes the page's raw HTML string into that file. The result: a local `.html` file you can reopen anytime without re-downloading from the internet.

2.5 Downloading Many Pages in a Loop

Bulk Download with a For-Loop

```
1 for i in range(1, 51):
2     a = requests.get(f"https://books.toscrape.com/catalogue/page-{i}.html")
3     with open(f"htmls/page{i}.html", "w", encoding="utf-8") as f:
4         f.write(a.text)
5         print(f"Downloaded page {i} successfully")
```

Line-by-Line Explanation

- **Line 1:** `range(1, 51)` generates the numbers 1 through 50 (51 is excluded). The loop variable `i` takes each of these values in turn — this lets you visit all 50 catalogue pages of the bookstore site one by one.
- **Line 2:** The f-string `f"...page-{i}.html"` substitutes the current value of `i` into the URL, so on the first loop it requests `page-1.html`, then `page-2.html`, and so on up to `page-50.html`.
- **Line 3:** Similarly, the output filename uses `i` so each page is saved with a unique name (`page1.html`, `page2.html`, ...) instead of overwriting the same file every loop. `encoding="utf-8"` is added here to safely handle any special characters (like the £ currency symbol on the book prices) that plain ASCII can't represent.
- **Line 4:** Writes that page's HTML into the newly opened file.
- **Line 5:** Prints a progress message so you can watch the download happen in real time — useful for confirming the loop is working and for knowing where it stopped if it ever fails partway through.

Why Loop Over Pages Like This?

Sites like `books.toscrape.com` paginate their listings — page 1 shows the first 20 books, page 2 the next 20, and so on. Downloading every page first (and saving locally) means you only hit the network *once* per page. Later, when writing and debugging your *parsing* logic with BeautifulSoup, you can re-read these saved files instantly instead of re-downloading from the internet every time you test your code.

3 The BeautifulSoup Module

What is BeautifulSoup?

BeautifulSoup is a Python library used to parse HTML and XML documents. It creates a **parse tree** from page content, making it easy to navigate and extract data. It is almost always used *after requests* — `requests` brings you the raw HTML text, and BeautifulSoup turns that text into something you can actually search and query, like a structured object instead of one giant blob of characters.

3.1 Installing and Creating a Soup Object

Install

```
1 pip install beautifulsoup4 lxml
```

Line-by-Line Explanation

- Installs two packages: `beautifulsoup4` (the BeautifulSoup library itself) and `lxml` (a fast C-based parser that BeautifulSoup can use internally).
- You technically don't *need* `lxml` — BeautifulSoup ships with a built-in `"html.parser"` — but `lxml` is faster and more forgiving of messy/broken HTML, which real-world websites often have.

Building the Soup Object

```
1 from bs4 import BeautifulSoup
2 import requests
3
4 url = "https://example.com"
5 response = requests.get(url)
6
7 soup = BeautifulSoup(response.text, "lxml")
```

Line-by-Line Explanation

- **Line 1:** Imports the `BeautifulSoup` class from the `bs4` package (the actual import name is `bs4`, even though you installed `beautifulsoup4`).
- **Lines 4–5:** Same `requests` pattern as before — fetch the raw HTML of the target page.
- **Line 7:** This is the key step. `BeautifulSoup(response.text, "lxml")` takes two arguments: the raw HTML string to parse, and the name of the parser engine to use (`"lxml"` here, or `"html.parser"` as a built-in alternative). The result, stored in `soup`, is a navigable tree object — this is the object you will call all your search methods on.

Mental Model: The HTML Tree

Given this HTML:

```
1 <html>
2   <body>
3     <h1>Title</h1>
4     <p class="description">This is a paragraph.</p>
5     <a href="/page">Read more</a>
6   </body>
7 </html>
```

BeautifulSoup treats it as a tree: `html` is the root, `body` is its child, and `h1`, `p`, `a` are children of `body`. Every tag can have attributes (like `class` or `href`) and text content. Searching with BeautifulSoup means walking and filtering this tree.

3.2 Reading Saved HTML and Parsing It

Loading a Local File into Soup

```
1 from bs4 import BeautifulSoup
2
3 with open("htmls/page1.html") as f:
4     content = f.read()
5
6 soup = BeautifulSoup(content, "html.parser")
```

Line-by-Line Explanation

- **Line 3:** Opens the previously saved HTML file (the one downloaded in the `requests` section) in **read mode** (the default mode, so no `"r"` flag is strictly needed).
- **Line 4:** `f.read()` reads the entire file content into one string, stored in `content` — this is identical in structure to what `response.text` would have given you directly from the network.
- **Line 6:** Builds the soup object exactly as before, but this time using `"html.parser"` (the parser built into Python, no extra installation needed) instead of `"lxml"`. Either parser works; `lxml` is just faster on large pages.

3.3 Accessing Elements — Dot Notation

Quick Access to First Match

```
1 soup.h1          # the first <h1> tag in the document
2 soup.h1.text     # just the text inside that tag
```

Line-by-Line Explanation

- `soup.h1` (dot notation) is a shortcut that returns the **first** occurrence of an `<h1>` tag anywhere in the document, including its full HTML (tags and all).
- `.text` strips away the tags and returns just the human-readable text content inside — this is usually what you actually want for data extraction.
- Dot notation only works for simple tag names and only finds the **first** match — for anything more specific, use `.find()` or `.select()` instead.

3.4 `find()` — One Match, With Filters

find Method

```
1 soup.find("p")
2 soup.find("p", class_="description")
```

Line-by-Line Explanation

- `soup.find("p")` returns the **first** `<p>` tag found, searched anywhere in the tree — same idea as `soup.p` but as an explicit method call, which makes the code more readable.
- `soup.find("p", class_="description")` adds a filter: only return a `<p>` tag whose `class` attribute equals `"description"`. Note the trailing underscore in `class_` — this is necessary because `class` alone is a reserved Python keyword and can't be used directly as an argument name.

3.5 find_all() — All Matches

find_all Method

```
1 h3s = soup.find_all("h3")
2 links = soup.find_all("a")
```

Line-by-Line Explanation

- `find_all("h3")` returns a **list** of every `<h3>` tag on the page, in the order they appear — not just the first one.
- This is the workhorse method for scraping: when a page has a repeating structure (e.g. a list of products, each wrapped in its own tag), `find_all` grabs every instance at once, ready to be looped over.

3.6 select() and select_one() — CSS Selectors

CSS Selector Methods

```
1 soup.select_one("p.description")
2 soup.select("a")
3 articles = soup.select("article.product_pod")
```

Line-by-Line Explanation

- `select_one(...)` and `select(...)` use **CSS selector syntax** — the same syntax used in web stylesheets — instead of separate tag/attribute arguments.
- `"p.description"` reads as "a `<p>` tag with class `description`" — the dot prefix means class. This is often shorter to write than the equivalent `find("p", class_="description")`.
- `select_one(...)` returns a single (first) match; `select(...)` (no `"_one"`) returns a list of **all** matches, same as `find_all`.
- `soup.select("article.product_pod")` selects every `<article class="product_pod">` block — on `books.toscrape.com`, this exact selector grabs every individual book listing on the page, each as one tree node you can then dig into further.

3.7 Extracting Attribute Values

Getting an href

```
1 link = soup.find("a")
2 print(link["href"])
3 print(link.get("href"))
```

Line-by-Line Explanation

- **Line 1:** Finds the first `<a>` (anchor/link) tag on the page.
- **Line 2:** `link["href"]` accesses the tag's `href` attribute like a dictionary key — this is how you get the actual URL out of a link tag, since `.text` would only give you the visible link text, not the destination.
- **Line 3:** `link.get("href")` does the same thing but is **safer**: if the attribute doesn't exist, `.get()` returns `None` instead of raising a `KeyError` and crashing your script.

3.8 Traversing the Tree

Parents, Children, Siblings

```
1 soup.p.parent
2 list(soup.body.children)
3 soup.h1.find_next_sibling()
```

Line-by-Line Explanation

- `.parent` moves **up** the tree to the tag that contains the current one.
- `.children` gives an iterator over all direct child tags of an element; wrapping it in `list(...)` turns that iterator into a normal Python list you can inspect or loop over.
- `.find_next_sibling()` moves **sideways** — it returns the next tag at the *same level* of the tree, immediately following the current one. Useful when data you need is positioned right after a tag you can easily identify (e.g. a label followed by its value).

3.9 Handling Missing Elements Safely

Defensive Scraping

```
1 title_tag = soup.find("h1")
2 if title_tag:
3     print(title_tag.text)
4 else:
5     print("Title not found")
```

Line-by-Line Explanation

- **Line 1:** `soup.find(...)` returns `None` if no matching tag exists anywhere on the page (it does **not** raise an error).
- **Lines 2–3:** Checking `if title_tag:` works because `None` is "falsy" in Python — so this safely guards against calling `.text` on something that doesn't exist, which would otherwise crash with an `AttributeError`.
- **Lines 4–5:** Falls back to a clear message instead of crashing — essential when scraping many pages in a loop, since one missing tag on one page shouldn't kill your entire scrape.

Always Inspect Before You Scrape

Before writing extraction code for a real site, open the page in your browser, right-click the element you want, and choose "Inspect" (or press F12) to see its actual tag name, classes, and structure. Never guess the HTML structure — verify it first.

4 Full Mini Project: Scraping Book Data to CSV

Putting It All Together

This section walks through a complete, real extraction project from raw saved HTML all the way to a clean CSV file — the same pipeline used for the `books.toscrape.com` practice site. Every line is explained individually below.

4.1 Step 1: Load the Saved Page and Build Soup

Setup

```
1 from bs4 import BeautifulSoup
2
3 with open("htmls/page1.html") as f:
4     content = f.read()
5
6 soup = BeautifulSoup(content, "html.parser")
```

Line-by-Line Explanation

- Same pattern covered earlier: read the locally-saved HTML file, then wrap its text content in a BeautifulSoup object so it becomes searchable.
- Working from the saved local file (instead of re-fetching with `requests` every time) makes development much faster, since you're not hitting the network on every test run while you're still figuring out your selectors.

4.2 Step 2: Select Every Book Block on the Page

Isolating Each Product

```
1 articles = soup.select("article.product_pod")
```

Line-by-Line Explanation

- On books.toscrape.com, every single book on a catalogue page is wrapped in `<article class="product_pod">...</article>`.
- `soup.select("article.product_pod")` grabs **all** of these blocks as a list. `articles` now holds, say, 20 separate mini-trees — one per book — each of which can be individually searched for its own title, price, and rating in the next step.
- This "isolate the repeating container first, then loop and dig into each one" is the single most common pattern in web scraping.

4.3 Step 3: Loop Over Each Book and Extract Fields

Extraction Loop

```
1 items = []
2 for article in articles:
3     title = article.find("h3").find("a")["title"]
4     price = article.select_one("p.price_color").text.split("$")[1]
5     rating_element = article.select_one("p.star-rating")
6     rating = rating_element['class'][1]
7     items.append([title, price, rating])
```

Line-by-Line Explanation

- **Line 1:** Creates an empty list, `items`, that will collect one row (one book's data) per loop iteration.
- **Line 2:** Loops over every book block found in Step 2. Each `article` is one book's isolated mini-tree.
- **Line 3 — Title:** Inside this `article`, `.find("h3")` finds the heading tag, `.find("a")` then

finds the link tag nested inside that heading, and `["title"]` reads that link's `title` attribute — on this site, the book's full title is stored as an attribute (not as visible text), because the visible link text is often truncated with "...".

- **Line 4 — Price:** `.select_one("p.price_color")` grabs the price tag. `.text` pulls out the visible text (something like "£51.77"). `.split("£")` breaks that string apart at the £ symbol, producing a list like `["", "51.77"]` — indexing with `[1]` takes the second piece, i.e. just the numeric price without the currency symbol.
- **Line 5 — Rating element:** `.select_one("p.star-rating")` grabs the tag that holds the star rating. Notably, this site doesn't put the rating in the visible text at all — it encodes it as a CSS class instead (e.g. `class="star-rating Three"`).
- **Line 6 — Rating value:** `rating_element['class']` returns the `class` attribute as a **list of strings** (an element can have multiple CSS classes, e.g. `["star-rating", "Three"]`). Indexing with `[1]` skips past the generic `"star-rating"` class and grabs the second one — the actual rating word like `"Three"`.
- **Line 7:** Appends a single list `[title, price, rating]` to `items` — this becomes one row of the final dataset. After the loop finishes, `items` holds a list of lists: one inner list per book.

Why Inspect the Class Attribute Like This?

Extracting data from a CSS class instead of visible text (as with the star rating) is a common scraping trick — but it's fragile. If the site's developers ever rename their CSS classes, this code breaks silently or throws an error. Always re-verify selectors if a scrape that used to work suddenly returns empty or wrong data.

4.4 Step 4: Structure the Data with pandas

Building a DataFrame

```
1 import pandas as pd
2
3 df = pd.DataFrame(items, columns=["Book Title", "Price", "Rating"])
4 df
```

Line-by-Line Explanation

- **Line 1:** Imports pandas, the standard library for tabular data in Python, conventionally aliased as `pd`.
- **Line 3:** `pd.DataFrame(items, columns=[...])` converts the list of lists (`items`) into a proper table. The `columns` argument names each column in order, matching the order fields were appended in Step 3 (`title`, `price`, `rating`).
- **Line 4:** Simply displaying `df` in a notebook renders it as a readable table — a good sanity check before saving, to confirm the data looks correct (no `Nones`, prices look numeric-ish, ratings look like real words).

4.5 Step 5: Save to CSV

Persisting the Dataset

```
1 df.to_csv("data.csv", index=False)
```

Line-by-Line Explanation

- `.to_csv("data.csv", ...)` writes the DataFrame out to a CSV file named `data.csv` in the current working directory.
- `index=False` tells pandas **not** to write its own internal row numbers (0, 1, 2, ...) as an extra column in the file — without this, you'd get an unwanted unnamed index column cluttering the CSV.
- At this point, the raw HTML page has been fully transformed into a clean, structured, reusable dataset — ready to be loaded back with `pd.read_csv(...)` for analysis, visualization, or modeling.

Scaling This to All 50 Pages

Combine this with the bulk-download loop from Section 2: instead of opening just `page1.html`, loop over `page1.html` through `page50.html`, repeat Steps 2–3 inside that loop, and keep appending to the **same** `items` list across all pages before building one final DataFrame at the end. This is exactly how you'd turn a 20-book single-page scrape into a full 1000-book catalogue dataset.

5 Extras: Beyond the Notebooks

Why This Section Exists

The notebooks covered the core `requests` + BeautifulSoup workflow. The topics below are commonly expected knowledge in data science interviews and real-world scraping projects, but weren't in the original material — included here as supplementary study notes.

5.1 Headers and User-Agents

Sending Custom Headers

```
1 headers = {  
2     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"  
3 }  
4 response = requests.get(url, headers=headers)
```

Line-by-Line Explanation

- By default, `requests` identifies itself with a User-Agent string that clearly says "python-requests" — many sites detect and block this automatically.
- Passing a `headers` dictionary with a browser-like `User-Agent` makes the request look like it's coming from a real browser, which can avoid simple bot-blocking (status 403).
- This is not a guaranteed bypass for serious anti-bot systems, just the first and most common fix when a request unexpectedly returns 403 Forbidden.

5.2 Query Parameters

Passing URL Parameters Cleanly

```
1 params = {"search": "laptop", "page": 2}  
2 response = requests.get("https://example.com/search", params=params)  
3 print(response.url)
```

Line-by-Line Explanation

- Instead of manually building a URL string like "...search?search=laptop&page=2", the `params` dictionary lets `requests` build and properly encode the query string for you (handling spaces, special characters, etc. correctly).
- `response.url` prints the final, fully-constructed URL that was actually requested — useful for debugging when results don't look right.

5.3 Handling Request Failures Gracefully

try/except with Timeouts

```
1 import time
2
3 try:
4     response = requests.get(url, timeout=5)
5     response.raise_for_status()
6 except requests.exceptions.RequestException as e:
7     print(f"Request failed: {e}")
```

Line-by-Line Explanation

- `timeout=5` tells `requests` to give up and raise an error if the server doesn't respond within 5 seconds, instead of hanging forever — essential when scraping many pages in a loop unattended.
- `response.raise_for_status()` explicitly raises an exception if the status code indicates an error (4xx or 5xx), turning a "silent bad response" into a catchable error.
- Wrapping in `try/except requests.exceptions.RequestException` catches **any** requests-related failure (connection errors, timeouts, bad status codes) in one place, letting your scraping loop continue to the next page instead of crashing entirely on one bad request.

5.4 Being a Polite Scraper: Rate Limiting

Adding Delays Between Requests

```
1 import time
2
3 for i in range(1, 51):
4     response = requests.get(f"https://example.com/page-{i}.html")
5     # ... process response ...
6     time.sleep(1)
```

Line-by-Line Explanation

- `time.sleep(1)` pauses execution for 1 second before the loop continues to the next iteration.
- Adding a small delay between requests prevents overwhelming the target server with rapid-fire traffic, reduces the chance of getting your IP rate-limited or banned, and is considered good scraping etiquette.
- For larger scrapes, a random delay (e.g. `time.sleep(random.uniform(1, 3))`) is often used instead of a fixed one, to look less like an automated, mechanical pattern.

5.5 Pagination Patterns

Two Common Pagination Styles

- **Numbered URL pagination** (as in `books.toscrape.com`): the page number is directly part of the URL, e.g. `/catalogue/page-1.html`, `/catalogue/page-2.html`. This is the easiest case — just loop and substitute the number, as already covered.
- **"Next" button pagination**: the page itself contains a link/button labeled "Next" that points to the next page's URL. Instead of guessing total page counts, you scrape the current page, then follow the "Next" link, and repeat until no "Next" link exists (commonly detected with `soup.select_one(".next a")` returning `None`).

5.6 When to Use an API Instead

Always Check for an API First

Many sites expose an official or semi-official JSON API that delivers the same data far more reliably than scraping HTML — and HTML structure changes frequently, breaking scrapers, while APIs are versioned and more stable. Before writing scraping code for a real site, check the browser's Network tab (F12 → Network → XHR/Fetch) while browsing the page — if data loads via background JSON requests, you can often call that same endpoint directly with `requests` and skip BeautifulSoup entirely.

Calling a JSON API Directly

```
1 response = requests.get("https://api.example.com/products")
2 data = response.json()
3 print(data[0]["name"])
```

Line-by-Line Explanation

- `response.json()` parses a JSON response body directly into a native Python data structure (a dict or list) — no BeautifulSoup needed at all, since JSON is already structured data, not HTML to be torn apart.
- Indexing into `data` (e.g. `data[0]["name"]`) then works just like any nested Python dictionary/list.

5.7 Dynamic (JavaScript-Rendered) Pages

The Limitation of `requests` + BeautifulSoup

`requests` only downloads the raw HTML the server initially sends — it does **not** run any JavaScript. Many modern sites load their actual content *after* the page loads, via JavaScript. If `response.text` looks empty or missing the data you can see in your browser, this is almost always why.

The Fix: Browser Automation

Tools like **Selenium** or **Playwright** launch an actual (often headless, i.e. invisible) browser, let the JavaScript run and the page fully render, and *then* let you extract the resulting HTML — at that point, you can still feed it into BeautifulSoup for the actual parsing step.

Selenium Skeleton — for Reference

```

1 from selenium import webdriver
2 from bs4 import BeautifulSoup
3
4 driver = webdriver.Chrome()
5 driver.get("https://example.com")
6
7 html = driver.page_source
8 soup = BeautifulSoup(html, "html.parser")
9
10 driver.quit()

```

Line-by-Line Explanation

- `webdriver.Chrome()` launches an actual Chrome browser instance under Python's control.
- `driver.get(url)` navigates that browser to the target page and waits for it to load, including running any JavaScript.
- `driver.page_source` returns the **fully rendered** HTML — including content that JavaScript added after the initial load — unlike `requests`, which only ever sees the original, unrendered HTML.
- From here, the same `BeautifulSoup` parsing techniques from earlier sections apply unchanged.
- `driver.quit()` closes the browser instance, freeing system resources — always call this when done, especially in loops, or you'll accumulate many open browser processes.

5.8 Quick Reference Table

Task	Tool / Method
Fetch raw HTML	<code>requests.get(url)</code>
Check success	<code>response.status_code == 200</code>
Parse HTML into a tree	<code>BeautifulSoup(html, "lxml")</code>
First match by tag	<code>soup.find("tag")</code>
First match, CSS selector	<code>soup.select_one("css")</code>
All matches by tag	<code>soup.find_all("tag")</code>
All matches, CSS selector	<code>soup.select("css")</code>
Get an attribute value	<code>tag["attr"]</code> or <code>tag.get("attr")</code>
Get visible text	<code>tag.text</code>
Structure into a table	<code>pd.DataFrame(rows, columns=[...])</code>
Save to file	<code>df.to_csv("file.csv", index=False)</code>
JS-rendered pages	Selenium / Playwright